

République du Sénégal
Université Iba Der Thiam de Thiès
UFR Sciences et Technologies
Département de Mathématiques et Informatique

Conception et Implémentation de Structures de Données

pour un Système de Gestion
de Dossiers Médicaux

Réalisé par :
Coumba NDIAYE
Moussa FAYE

Encadrant :
Dr Abdoulaye DIALLO

Licence 2 Mathématiques-Informatique
Année universitaire 2025–2026

Table des matières

1	Introduction	2
2	Modélisation des données	2
2.1	Structure Patient	2
2.2	Structure Consultation	2
2.3	Relation entre les structures	2
3	Structures de données implémentées	3
3.1	Tableau statique	3
3.1.1	Caractéristiques	3
3.1.2	Opérations	3
3.2	Tableau dynamique	3
3.2.1	Principe du redimensionnement	4
3.2.2	Caractéristiques	4
3.3	Liste chaînée	4
3.3.1	Structure d'un nœud	4
3.3.2	Caractéristiques	4
4	Analyse de complexité	4
4.1	Tableau récapitulatif	4
4.2	Justifications mathématiques	4
4.3	Discussion	5
5	Étude expérimentale	5
5.1	Protocole expérimental	5
5.2	Temps d'insertion	6
5.3	Temps de recherche	6
5.4	Temps de tri	6
5.5	Courbe théorique vs empirique	6
5.6	Comparaison globale	6
6	Conclusion	6

1 Introduction

Dans tout système informatique, le choix des structures de données conditionne directement la performance des opérations fondamentales. Ce projet s'inscrit dans le cadre du cours d'**Algorithmique et Structures de Données** de la Licence 2 Mathématiques-Informatique de l'Université Iba Der Thiam de Thiès.

Nous avons choisi le domaine de la **gestion de dossiers médicaux**, qui consiste à gérer les informations des patients et leurs consultations dans un centre de santé.

L'objectif principal est d'implémenter et de comparer trois structures de données en langage C :

- Le **tableau statique** : structure simple à taille fixe
- Le **tableau dynamique** : structure à redimensionnement automatique
- La **liste chaînée** : structure à allocation dynamique par nœud

Ce rapport présente la modélisation des données, l'implémentation des structures, une analyse de complexité théorique, et une étude expérimentale comparative appuyée par des courbes scientifiques.

2 Modélisation des données

2.1 Structure Patient

Champ	Description
int id	Identifiant unique (clé primaire)
char prenom[25]	Prénom du patient
char nom[25]	Nom de famille
char sexe	Sexe (M ou F)
char adresse[50]	Adresse complète
char numTel[15]	Numéro de téléphone
int age	Âge en années
Date date_naissance	Date de naissance
float poids	Poids en kilogrammes
float taille	Taille en mètres
struct consultation* consultations	Pointeur vers les consultations
int nb_consultations	Nombre de consultations

TABLE 1 – Champs de la structure `patient`

2.2 Structure Consultation

2.3 Relation entre les structures

Les deux structures sont liées par des pointeurs, formant une relation **1-N** : un patient peut avoir plusieurs consultations, et chaque consultation référence son patient.

Champ	Description
int id	Identifiant unique
Date date_consultation	Date de la consultation
char medecin[25]	Nom du médecin
char diagnostic[100]	Diagnostic établi
char traitement[100]	Traitement prescrit
int duree	Durée en jours
float cout	Coût en FCFA
struct patient* patient	Pointeur vers le patient

TABLE 2 – Champs de la structure `consultation`

3 Structures de données implémentées

3.1 Tableau statique

Tableau de taille fixe défini à la compilation, pouvant contenir au maximum **100 patients**.

3.1.1 Caractéristiques

- Taille fixe de 100 éléments
- Accès direct par indice en $O(1)$
- Aucune gestion dynamique de la mémoire

3.1.2 Opérations

- **Insertion** : ajout à la position `taille`
- **Recherche par clé** : parcours séquentiel
- **Recherche par intervalle** : patients entre deux âges
- **Recherche par préfixe** : patients par début de nom
- **Suppression** : remplacement par le dernier élément
- **Modification** : mise à jour par identifiant
- **Tri par insertion** : tri alphabétique sur le nom
- **Tri rapide** : tri numérique sur l'âge
- **Agrégations** : min, max, moyenne, médiane, écart-type
- **Persistence** : sauvegarde et chargement binaire

3.2 Tableau dynamique

Utilise `malloc` et `realloc`. La capacité **double automatiquement** lorsqu'elle est atteinte.

3.2.1 Principe du redimensionnement

```
if (*taille >= *capacite) {  
    *capacite *= 2;  
    *tableau = realloc(*tableau, (*capacite) * sizeof(patient));  
}
```

3.2.2 Caractéristiques

- Capacité initiale de 2 éléments
- Facteur de croissance géométrique de 2
- Accès direct par indice en $O(1)$
- Libération mémoire avec `free()`

3.3 Liste chaînée

Structure où chaque nœud contient les données et un pointeur vers le nœud suivant.

3.3.1 Structure d'un nœud

```
typedef struct Noeud {  
    patient data;  
    struct Noeud* suivant;  
} Noeud;
```

```
typedef struct {  
    Noeud* tete;  
    int taille;  
} ListeChaine;
```

3.3.2 Caractéristiques

- Insertion en tête en $O(1)$
- Insertion en queue en $O(n)$
- Aucune limite de taille
- Allocation mémoire individuelle par nœud

4 Analyse de complexité

4.1 Tableau récapitulatif

4.2 Justifications mathématiques

Insertion en $O(1)$: L'ajout se fait directement à la position `taille` sans décalage. Pour le tableau dynamique, le coût amorti reste $O(1)$ car le doublement ne survient qu'en

Opération	Tableau statique	Tableau dynamique	Liste chaînée
Insertion	$O(1)$	$O(1)$ amorti	$O(1)$ tête
Recherche par clé	$O(n)$	$O(n)$	$O(n)$
Suppression	$O(n)$	$O(n)$	$O(n)$
Modification	$O(n)$	$O(n)$	$O(n)$
Tri par insertion	$O(n^2)$	$O(n^2)$	$O(n^2)$
Tri rapide	$O(n \log n)$	$O(n \log n)$	—
Agrégations	$O(n)$	$O(n)$	$O(n)$
Espace	$\Theta(1)$ fixe	$\Theta(n)$	$\Theta(n)$ + pointeurs

TABLE 3 – Complexités des trois structures

$O(\log n)$ occasions sur n insertions.

Tri par insertion en $O(n^2)$:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$$

Dans le meilleur cas (tableau déjà trié) : $\Omega(n)$.

Tri rapide en $O(n \log n)$:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \Rightarrow \Theta(n \log n)$$

Dans le pire cas : $O(n^2)$.

4.3 Discussion

Pour le tri rapide, le choix du dernier élément comme pivot conduit au pire cas $O(n^2)$ sur des données déjà triées. Sur des données uniformément distribuées, le comportement moyen $\Theta(n \log n)$ est observé.

5 Étude expérimentale

5.1 Protocole expérimental

Paramètre	Valeur
Système d'exploitation	Windows 11
Compilateur	GCC 16.1 (MinGW)
Répétitions par mesure	10
Tailles (statique)	$n = 10, 50, 100$
Tailles (dynamique & liste)	$n = 100, 1\,000, 10\,000$

TABLE 4 – Conditions expérimentales

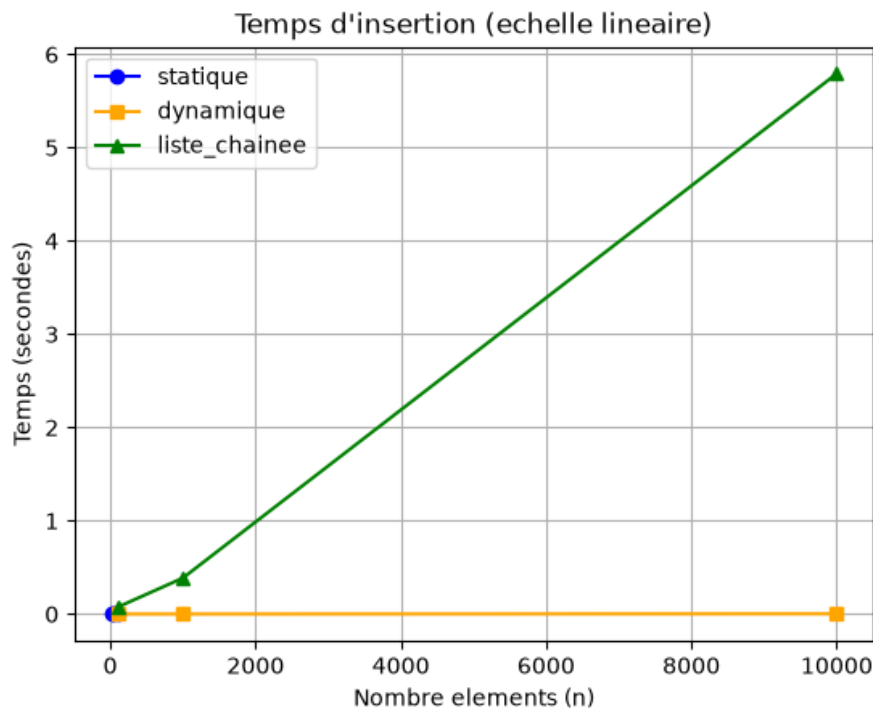


FIGURE 1 – Temps d’insertion en échelle linéaire

5.2 Temps d’insertion

La liste chaînée est plus lente que les tableaux pour l’insertion, en raison de l’appel à `malloc` pour chaque nœud.

5.3 Temps de recherche

5.4 Temps de tri

5.5 Courbe théorique vs empirique

La courbe empirique suit bien la courbe théorique $O(n \log n)$, confirmant notre analyse de complexité.

5.6 Comparaison globale

L’histogramme confirme que la liste chaînée est la structure la plus lente. Le tableau dynamique et le tableau statique présentent des performances comparables.

6 Conclusion

Ce projet nous a permis de mettre en pratique les notions fondamentales de l’algorithmique et des structures de données à travers un cas concret de gestion de dossiers médicaux.

Les principaux enseignements sont :

- Le **tableau statique** est le plus rapide pour les petits volumes, mais limité à 100

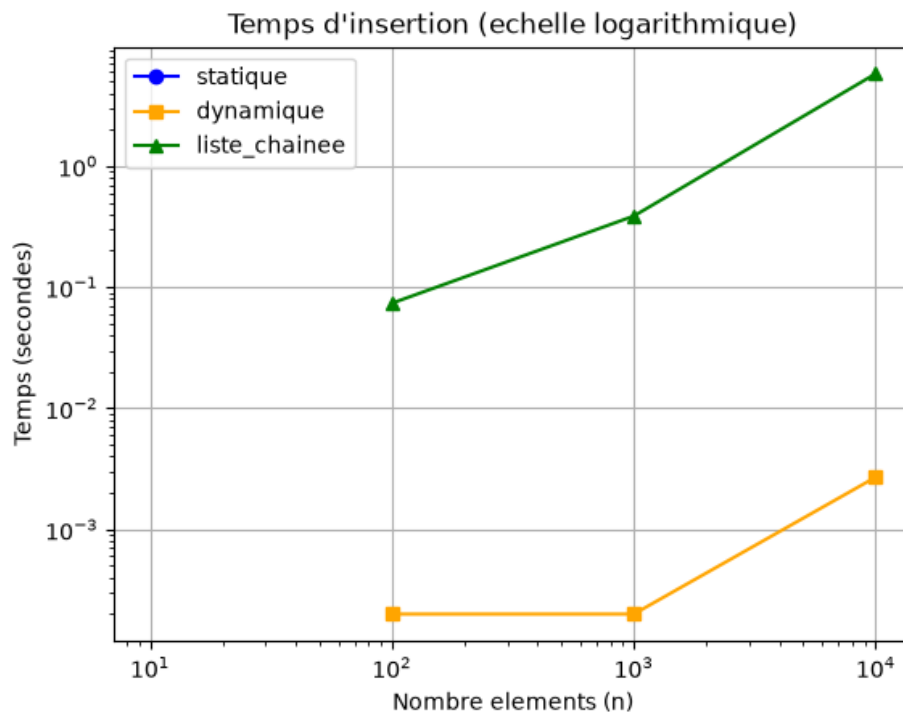


FIGURE 2 – Temps d’insertion en échelle logarithmique

éléments.

- Le **tableau dynamique** offre le meilleur compromis entre performance et flexibilité.
- La **liste chaînée** offre une insertion en tête en $O(1)$ et une taille illimitée, au prix d’une performance moindre en pratique.

Ce travail nous a permis de renforcer notre maîtrise des **pointeurs en C**, de la **gestion mémoire dynamique**, et des outils professionnels tels que Git, GCC et Python.

Références

- [1] Cormen T. H. et al., *Introduction to Algorithms*, 4^e édition, MIT Press, 2022.
- [2] Kernighan B. W., Ritchie D. M., *The C Programming Language*, 2^e édition, Prentice Hall.
- [3] Sedgewick R., Wayne K., *Algorithms*, 4^e édition, Addison-Wesley, 2011.
- [4] Beauquier D., Berstel J., Chrétienne P., *Éléments d’algorithmique*, Masson.

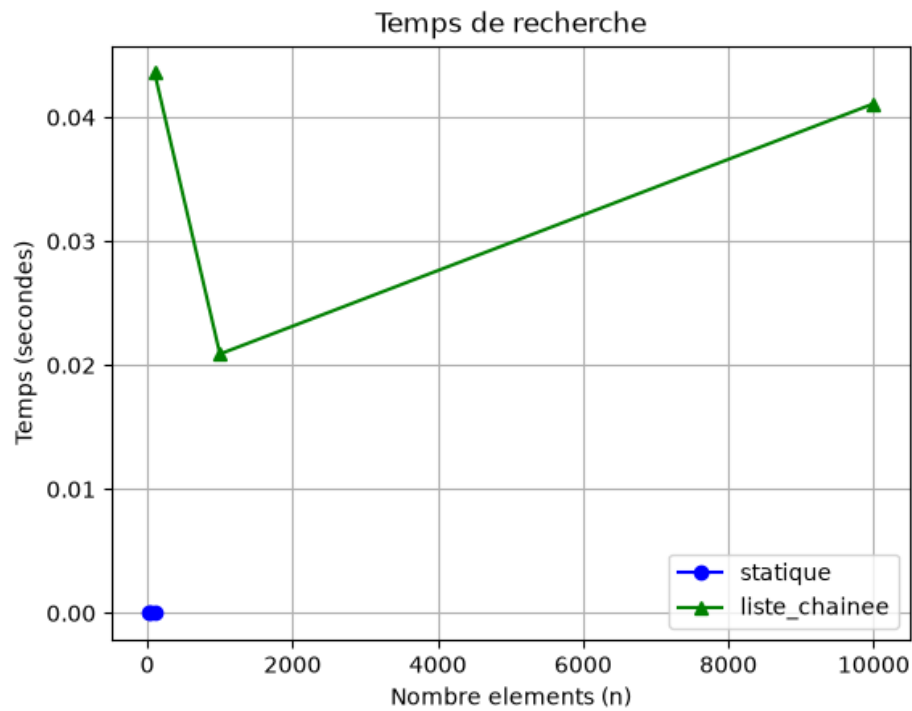


FIGURE 3 – Temps de recherche par clé pour chaque structure

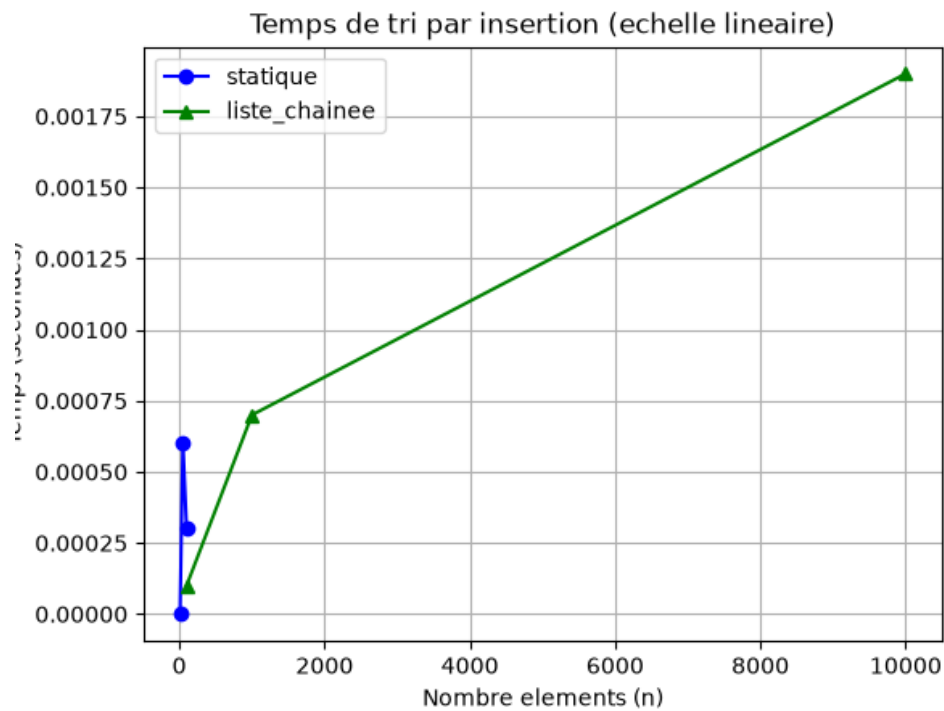


FIGURE 4 – Temps de tri par insertion en échelle linéaire

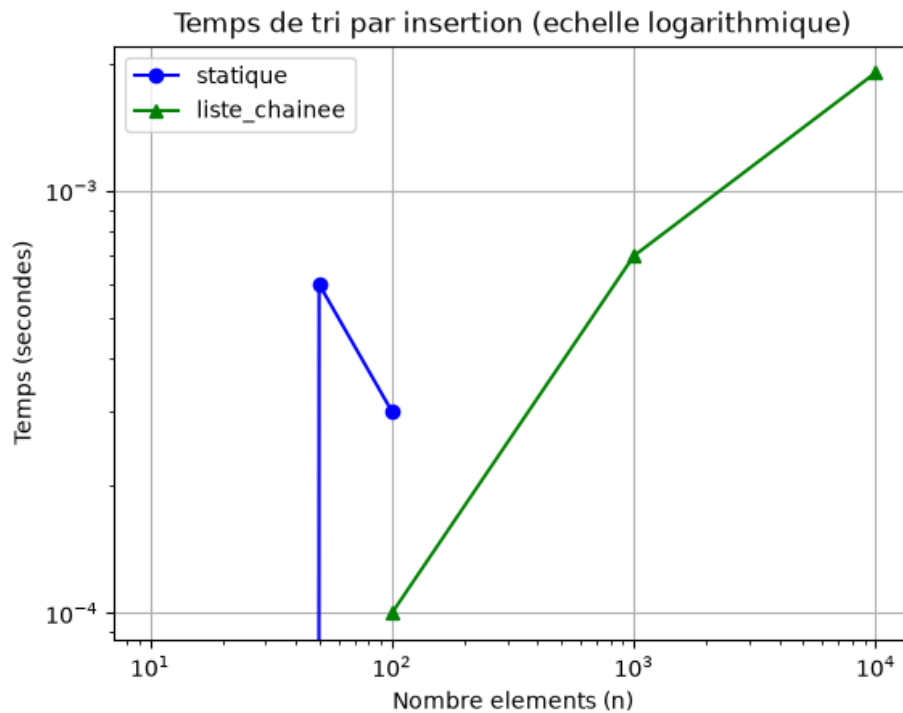


FIGURE 5 – Temps de tri par insertion en échelle logarithmique

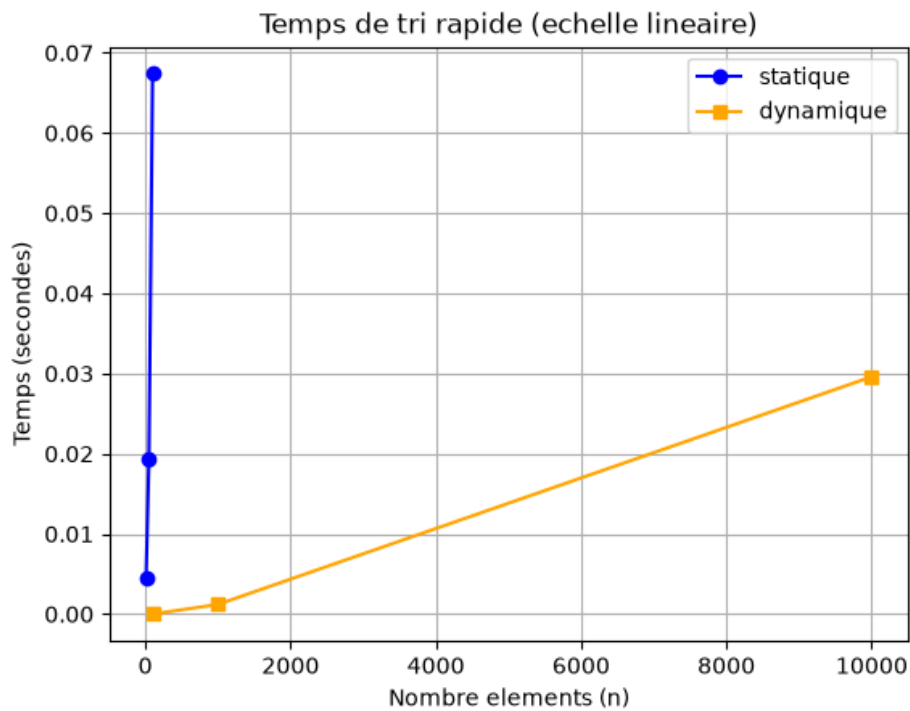


FIGURE 6 – Temps de tri rapide en échelle linéaire

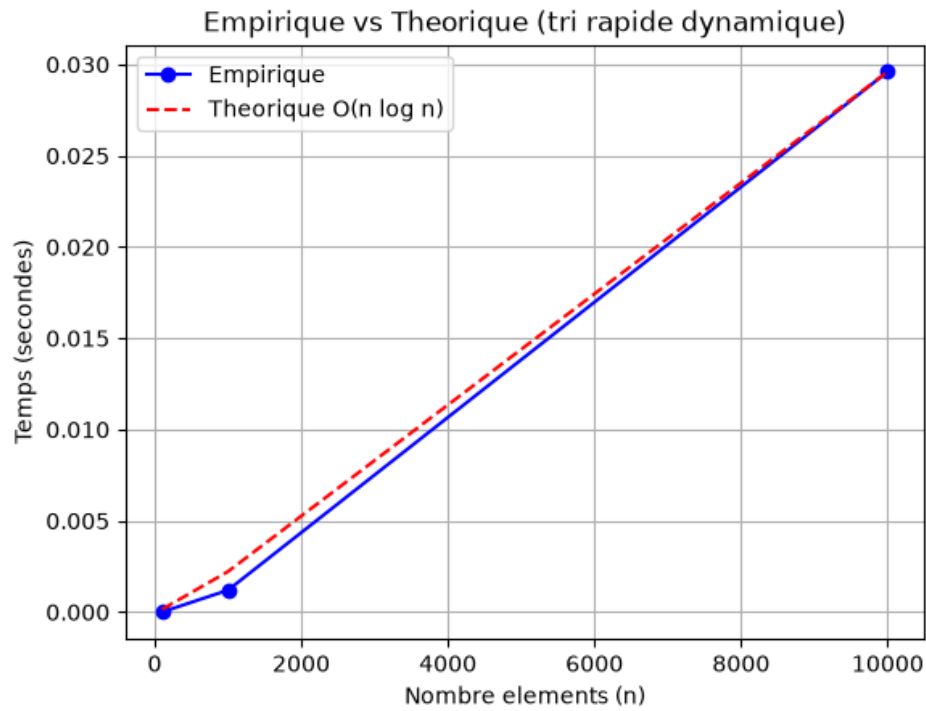
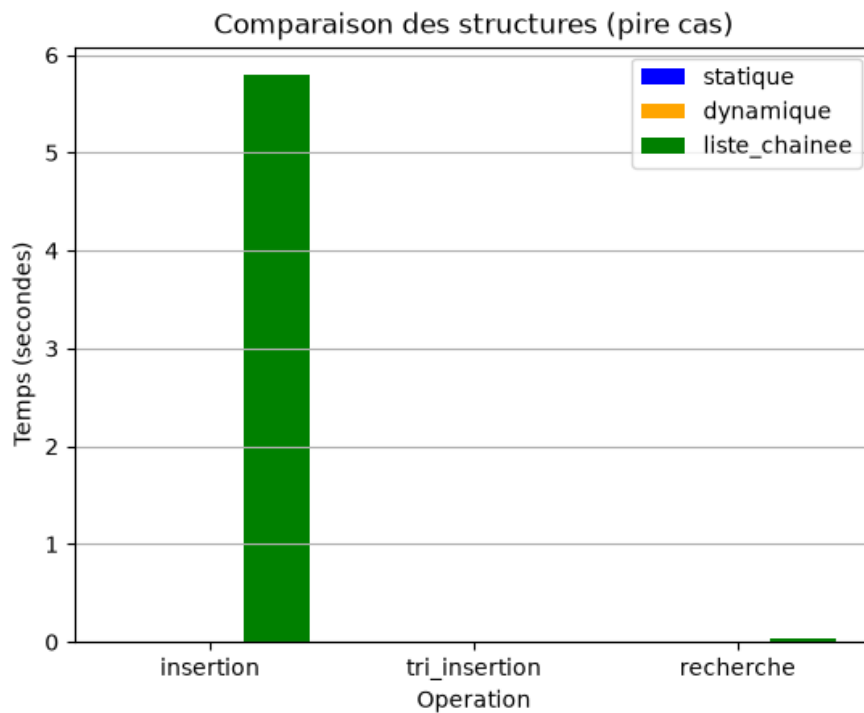
FIGURE 7 – Courbe empirique vs théorique $O(n \log n)$ 

FIGURE 8 – Histogramme comparatif des trois structures